

תרגילי PowerShell – יסודות

ארבעה פרקים, מהקל לקשה. כל פרק מתחיל בתרגילי חימום ומסתיים בתרגיל מסכם. הפתרונות נמצאים בקובץ `Solutions.ps1` באותה תיקייה – נסו לפתור לבד לפני שאתם מציצים.

הכנה: צרו תיקיית עבודה והיכנסו אליה. כל התרגילים עובדים בתוכה כדי לא ללכלך את המחשב.

```
New-Item -ItemType Directory -Path C:\PSLab -Force
Set-Location C:\PSLab
```

סימון רמת קושי: קל · בינוני · מאתגר

פרק 1 – פקודות בסיסיות ופרמטרים

1.1 קל מבנה הפקודה (Verb-Noun)

1. הריצו `Get-Verb` וספרו כמה פעלים מאושרים יש.
2. מצאו את כל הפקודות שה-Noun שלהן הוא `Service`.
3. מצאו את כל הפקודות שמתחילות ב-`Get-` ומכילות את המילה `Process`.

1.2 קל מערכת העזרה

1. הציגו את העזרה של `Get-ChildItem`.
2. הציגו רק את הדוגמאות (Examples) של `Get-ChildItem`.
3. הציגו את המידע המלא על הפרמטר `Filter` של `Get-ChildItem` בלבד.
4. חפשו בעזרה את כל הנושאים שמכילים את המילה `about_Parameters` וקראו אותו.

1.3 קל Aliases

1. גלו לאיזו פקודה מפנים ה-aliases הבאים: `ls`, `dir`, `cd`, `cat`, `?`, `%`.
2. מצאו את כל ה-aliases של `Get-ChildItem`.
3. צרו alias בשם `np` שפותח את `notepad.exe`, ובדקו שהוא עובד.

1.4 בינוני סוגי פרמטרים

לכל אחת מהפקודות הבאות כתבו **שתי גרסאות** – אחת עם פרמטרים בשם (Named) ואחת עם פרמטרים לפי מיקום (Positional):

1. הצגת תוכן התיקייה `C:\Windows`.
2. הצגת 5 האירועים האחרונים מלוג `System` (`Get-EventLog` ו-`Get-WinEvent`).
- אחר כך:
3. הריצו `Get-ChildItem C:\Windows` עם פרמטר מסוג **Switch** שמציג רק תיקיות.
4. הריצו אותו שוב כך שיציג רק קבצים, כולל קבצים מוסתרים.

5. בעזרת `Path` `Parameter` `Get-ChildItem` `Get-Help` בדקו: האם `-Path` חובה? מה המיקום (`Position`) שלו? האם הוא מקבל קלט מה-Pipeline?

1.5 בינוני פרמטרים משותפים (Common Parameters)

1. צרו 3 קבצים ריקים: `a.txt`, `b.txt`, `c.txt`.
2. נסו למחוק את כל קבצי ה-`txt` עם `-WhatIf`. מה קרה? האם הקבצים נמחקו?
3. מחקו רק את `a.txt` עם `-Confirm` וענו "כן".
4. נסו לקרוא קובץ שלא קיים (`Get-Content nothing.txt`) כך שלא תופיע שגיאה על המסך.
5. הריצו את אותה פקודה עם `ErrorVariable myErr` והציגו את תוכן `$myErr`.
6. העתיקו את `b.txt` ל-`b2.txt` עם `-Verbose` ושימו לב לפלט.

1.6 בינוני Parameter Sets

1. הריצו `Syntax` `Get-Process` `Get-Command`. כמה `Parameter Sets` יש לפקודה?
2. נסו להריץ `Get-Process -Id 4 -Name explorer`. למה זה נכשל?
3. מצאו את ההתליך שה-`Id` שלו הוא ה-`PID` של ה-`PowerShell` הנוכחי (רמז: `$PID`).

1.7 מאתגר Splatting

1. שמרו ב-`Hashtable` בשם `$params` את הפרמטרים: `File = , Filter = '*.exe' , Path = 'C:\Windows'`.
`$true`
2. הריצו `Get-ChildItem` עם ה-`(Splatting)` `Hashtable`.
3. הוסיפו ל-`Hashtable` את `ErrorAction = 'SilentlyContinue'` ו-`Recurse = $true` והריצו שוב.

1.8 מאתגר תרגיל מסכם – פונקציה עם פרמטרים

כתבו פונקציה `Get-BigFile` עם הפרמטרים:

פרמטר	סוג	דרישות
<code>Path</code>	<code>string</code>	חובה, <code>Position 0</code> , חייב להיות תיקייה קיימת (<code>ValidateScript</code>)
<code>MinSizeMB</code>	<code>int</code>	ברירת מחדל 10, בטווח 1–10000 (<code>ValidateRange</code>)
<code>Extension</code>	<code>string</code>	אופציונלי, רק אחד מ: <code>.log</code> , <code>.txt</code> , <code>.zip</code> , <code>.iso</code> (<code>ValidateSet</code>)
<code>Recurse</code>	<code>switch</code>	אם דולק – חיפוש בתתי-תיקיות

הפונקציה מחזירה את שם הקובץ, הנתיב והגודל ב-MB (מעוגל ל-2 ספרות), ממוינים מהגדול לקטן. הוסיפו `[CmdletBinding()] Write-Verbose -I` שמדפיס כמה קבצים נמצאו, ובדקו שהוא עובד עם `-Verbose`.

פרק 2 – אובייקטים

2.1 קל הכל אובייקט

1. שמרו את `Get-Date` במשתנה `$now`. הציגו רק את השנה, את היום בשבוע ואת השעה.
2. הריצו `$now | Get-Member`. מה ה-`TypeName` של האובייקט?
3. השתמשו במתודה של `$now` כדי לחשב את התאריך בעוד 100 יום.
4. חשבו כמה ימים עברו מאז 1 בינואר של השנה הנוכחית (רמז: חיסור תאריכים מחזיר `TimeSpan`).

2.2 קל Properties מול Methods

1. שמרו את התהליך של ה-`PowerShell` הנוכחי במשתנה `$me`.
2. הציגו את `Id`, `ProcessName` ואת צריכת הזיכרון (`WorkingSet64`) ב-MB.
3. בעזרת `Get-Member -MemberType Method` מצאו איזו מתודה עוצרת תהליך (אל תריצו אותה על `$me`).
4. לגבי מחרוזת `'PowerShell Is Great'`: הפכו אותה לאותיות גדולות, החליפו `Great` ב-`Awesome`, ופצלו אותה למערך מילים. כמה מילים יש?

2.3 בינוני Select-Object

1. הציגו את 5 התהליכים שצורכים הכי הרבה CPU, רק עם השדות `Name`, `Id`, `CPU`.
2. הציגו את אותם תהליכים עם שדה מחושב (Calculated Property) בשם `MemoryMB`.
3. בעזרת `ExpandProperty` קבלו רשימה של שמות השירותים בלבד (מחרוזות, לא אובייקטים). בדקו עם `Get-Member` את ההבדל בין `Select-Object Name` לבין `Select-Object -ExpandProperty Name`.
4. הציגו את כל ה-`Properties` של תהליך אחד (`Select-Object *`).

2.4 בינוני יצירת אובייקטים משלכם

1. צרו אובייקט `[PSCustomObject]` שמתאר שרת: `IsOnline`, `OS`, `IP`, `Name`.
2. צרו מערך של 4 שרתים כאלה (לפחות אחד `IsOnline = $false`).
3. הציגו רק את השרתים שאינם `Online`.
4. הוסיפו לכל השרתים `Property` חדש בשם `Checked` עם הזמן הנוכחי (`Add-Member`).
5. הוסיפו לאובייקט אחד `ScriptMethod` בשם `Describe` שמחזיר מחרוזת כמו `"SRV01 (10.0.0.1) - Online"`.

2.5 בינוני מערכים ו-Hashtables

1. צרו מערך של המספרים 1 עד 20. הציגו רק את הזוגיים.
2. חשבו סכום, ממוצע, מקסימום ומינימום בעזרת `Measure-Object`.
3. צרו `Hashtable` של שם משתמש → מחלקה (לפחות 4 רשומות). הוסיפו רשומה, מחקו רשומה, ובדקו אם מפתח קיים.
4. המירו את ה-`Hashtable` לאובייקט (`[PSCustomObject]`) והשוו את הפלט של `Get-Member` לשניהם.

2.6 מאתגר Sort-Object ו-Group-Object

1. קבצו את השירותים במחשב לפי `Status`. כמה רצים וכמה עצורים?
2. קבצו את הקבצים ב-`C:\Windows\System32` לפי סיומת, והציגו את 10 הסיומות הנפוצות ביותר.

3. לכל קבוצה מסעיף 2 חשבו גם את הגודל הכולל ב-MB.
4. מיינו את התהליכים לפי Company ואחר כך לפי CPU בסדר יורד.

2.7 מאתגר תרגיל מסכם – דוח מערכת

כתבו סקריפט שבונה אובייקט אחד בשם `$report` עם השדות:

- `ComputerName`
 - OS (שם מערכת ההפעלה – `Get-CimInstance Win32_OperatingSystem`)
 - `UptimeHours` (מעוגל)
 - `CPUCount`
 - `FreeRamGB / TotalRamGB`
 - `TopProcesses` – מערך של 3 התהליכים הכבדים ביותר בזיכרון (שם + MB)
 - `StoppedAutoServices` – שירותים שמוגדרים `Automatic` אבל לא רצים
- הציגו את `$report` ואת `$report.TopProcesses` בצורה מסודרת.

פרק 3 – Pipeline-Input, Output

3.1 קלט מהמשתמש

1. בקשו מהמשתמש את שמו (Read-Host) והדפיסו "<שם> שלום", השנה עכשיו <שעה>".
2. בקשו סיסמה כך שלא תוצג על המסך (Read-Host -AsSecureString). מה הסוג של המשתנה שחזר?
3. בקשו מהמשתמש שני מספרים והדפיסו את הסכום. שימו לב: מה קורה אם לא ממירים ל-[int] ?

3.2 קלט פקודות הפלט השונות

הריצו כל אחת מהשורות וכתבו לעצמכם מה ההבדל:

```
Write-Output "output"
Write-Host "host" -ForegroundColor Green
Write-Verbose "verbose"
Write-Warning "warning"
Write-Error "error"
Write-Information "info"
```

1. אילו מהן לא הודפסו? איך גורמים להן להופיע (InformationAction, \$VerbosePreference)?
2. הריצו \$x = Write-Output "a" ו-\$y = Write-Host "b". מה נמצא ב-\$x ומה ב-\$y? למה?

3.3 בינוני פורמט הפלט

1. הציגו את השירותים כטבלה (Format-Table) עם Name, Status, StartType ו-AutoSize.
2. הציגו תהליך אחד כרשימה (Format-List) עם כל השדות.
3. הציגו את שמות השירותים בעמודות (Format-Wide -Column 4).
4. הציגו את התהליכים בחלון גרפי עם אפשרות לבחור שורות (Out-GridView -PassThru), ועצרו את מה שבחרתם עם -WhatIf.
5. שאלת הבנה: למה Get-Service | Format-Table | Sort-Object Name לא עובד כמו שמצפים? מה הכלל?

3.4 בינוני Pipeline – סינון ועיבוד

1. הציגו רק שירותים שרצים ושמן מתחיל ב-W – פעם עם Where-Object בכתוב מלא ({ \$_.Status ... }) ופעם בכתוב המקוצר (Where-Object Status -eq ...).
2. עםForEach-Object הדפיסו לכל קובץ בתיקייה הנוכחית: "bytes <שם> - <גודל>".
3. חשבו את הגודל הכולל של כל קבצי ה-log ב-C:\Windows (כולל תתי-תיקיות) ב-MB.
4. השתמשו ב-ForEach-Object עם -Begin, -Process, -End כדי לספור כמה קבצים עברו ב-Pipeline ולהדפיס את הסיכום בסוף.

3.5 בינוני איך ה-Pipeline מחבר פרמטרים

1. הריצו Get-Help Stop-Service -Parameter Name ו-Get-Help Stop-Service -Parameter InputObject. מי מקבל ByValue ומי ByPropertyName?

2. צרו קובץ names.txt עם השורות Spooler ו-W32Time. הריצו `Get-Service names.txt | Get-Content names.txt`. למה זה עובד?
3. צרו אובייקטים עם Property בשם Name (לדוגמה `[PSCustomObject]@{Name='Spooler'}`) ושלחו אותם ל-`Get-Service`. איזה סוג Binding עבד כאן?
4. **Trace**: הריצו `Trace-Command -Name ParameterBinding -Expression { 'Spooler' | Get-Service }`. PSHost וחפשו בפלט איך הפרמטר קושר.

3.6 בינוני הפניית פלט (Redirection)

- שמרו את רשימת התהליכים לקובץ `procs.txt` עם `>` ואחר כך עם `.Out-File`. מה ההבדל בקידוד?
- הוסיפו לסוף הקובץ את התאריך עם `>>`.
- הריצו פקודה שמייצרת גם פלט וגם שגיאה (`Get-ChildItem C:\Windows, C:\NoSuchFolder`) ושמרו **רק את השגיאות** בקובץ `errors.txt` (`>> 2`).
- שמרו את הכל (פלט + שגיאות) לאותו קובץ (`>> *`).
- השתמשו ב-`Tee-Object` כדי גם להציג על המסך וגם לשמור לקובץ.

3.7 מאתגר פונקציה שמקבלת Pipeline

כתבו פונקציה `Test-Port` שמקבלת:

- `ComputerName` – מה-Pipeline (`ValueFromPipeline` וגם `ValueFromPipelineByPropertyName`), תומכת במערך.
- `Port` – ברירת מחדל 443.

הפונקציה מחזירה אובייקט עם `Open`, `Port`, `ComputerName` (True/False) לכל מחשב. השתמשו בבלוקים `begin/process/end`. בדקו שכל אלה עובדים:

```
Test-Port -ComputerName google.com, localhost
'google.com','localhost' | Test-Port -Port 80
Import-Csv servers.csv | Test-Port          # CSV עם עמודה ComputerName
```

(רמז: `Test-NetConnection -InformationLevel Quiet` ו-`System.Net.Sockets.TcpClient`)

פרק 4 – עבודה עם קבצים בפורמטים שונים

4.1 קל קבצי טקסט

1. צרו קובץ `servers.txt` עם 5 שמות שרתים, שורה לכל שרת (`Set-Content`).
2. הוסיפו שרת שישי (`Add-Content`).
3. קראו את הקובץ והציגו: את מספר השורות, את השורה הראשונה, את 2 השורות האחרונות (`-TotalCount`), `-Tail`.
4. קראו את הקובץ כמחרוזת אחת (`-Raw`) והשוו את ה-`Type` לקריאה רגילה.
5. חפשו בקובץ שורות שמכילות `SRV` בעזרת `Select-String`, והציגו את מספר השורה של כל התאמה.

4.2 בינוני עיבוד לוג טקסט

צרו קובץ `app.log` עם התוכן הבא:

```
2026-09-01 10:00:01 INFO User david logged in
2026-09-01 10:02:15 ERROR Database timeout on SRV-DB01
2026-09-01 10:05:40 WARN Disk C: 85% on SRV-APP02
2026-09-01 10:07:12 ERROR Database timeout on SRV-DB01
2026-09-01 10:09:55 INFO User dana logged out
2026-09-01 10:11:30 ERROR Service crashed on SRV-APP01
```

1. ספרו כמה שורות מכל רמה (`ERROR / WARN / INFO`).
2. חלצו מכל שורת `ERROR` את שם השרת בעזרת `Regex` (`-match` -`$Matches`).
3. המירו כל שורה לאובייקט עם `Date`, `Level`, `Message` (השתמשו ב-`-split` או ב-`Regex` עם קבוצות בשם).
4. שמרו את שורות ה-`ERROR` בלבד לקובץ `errors_only.log`.

4.3 בינוני CSV

1. ייצאו את רשימת השירותים (`Name`, `DisplayName`, `Status`, `StartType`) לקובץ `services.csv` ללא שורת הטיפוס (`-NoTypeInfo`).
2. פתחו את הקובץ ב-`Excel/Notepad` ובדקו איך הוא נראה.
3. ייבאו את הקובץ חזרה (`Import-Csv`) והציגו רק את השירותים העצורים. **שימו לב:** מה ה-`Type` של `Status` אחרי הייבוא?
4. צרו ידנית קובץ `users.csv`:

```
Name,Department,Salary
David,IT,18000
Dana,HR,15000
Yossi,IT,21000
Rina,Finance,17500
```

- ייבאו אותו, חשבו ממוצע שכר לכל מחלקה (שימו לב – צריך להמיר את `Salary` למספר).
5. הוסיפו לכל משתמש עמודה חדשה `Email` בפורמט `name@company.local` ושמרו לקובץ חדש.

6. ייצאו קובץ CSV עם מפריד ; (Delimiter) וייבאו אותו חזרה נכון.

4.4 JSON בינוני

1. המירו את 3 התהליכים הראשונים (CPU, Id, Name) ל-JSON והציגו על המסך.
2. שמרו את מערך השרתים מתרגיל 2.4 לקובץ servers.json.
3. קראו את servers.json חזרה לאובייקטים (Get-Content -Raw | ConvertFrom-Json) והציגו רק את השרתים Online-ה.
4. צרו קובץ הגדרות config.json:

```
{
  "AppName": "Monitor",
  "Version": "1.2",
  "Servers": [
    { "Name": "SRV01", "Port": 443 },
    { "Name": "SRV02", "Port": 8080 }
  ],
  "Settings": { "Retries": 3, "TimeoutSec": 30 }
}
```

- קראו אותו, שנו את Retries ל-5, הוסיפו שרת SRV03 בפורט 22, ושמרו חזרה.
5. **שאלת הבנה:** מה עושה הפרמטר Depth -ב- ConvertTo-Json? הדגימו מקרה שבו בלעדיו מאבדים מידע.

4.5 XML מאתגר

1. ייצאו את התהליכים ל-XML בעזרת Export-Clixml וייבאו חזרה. מה ההבדל בין האובייקט שחזר לאובייקט המקורי? (בדקו עם Get-Member)
2. צרו קובץ inventory.xml:

```
<Inventory>
  <Server name="SRV01" role="Web" ram="16" />
  <Server name="SRV02" role="DB" ram="64" />
  <Server name="SRV03" role="Web" ram="8" />
</Inventory>
```

- טענו אותו עם [xml] והציגו את כל השרתים מסוג Web.
3. מצאו את השרתים עם יותר מ-10GB RAM בעזרת XPath-ו Select-Xml.
 4. הוסיפו לקובץ שרת חדש SRV04 (RAM, App=Role=32) ושמרו (.Save()).
 5. שמרו Credential מוצפן בקובץ (cred.xml | Export-Clixml | Get-Credential) וטענו אותו חזרה. על איזה מחשב/משתמש אפשר לפענח אותו?

4.6 HTML ופורמטים נוספים מאתגר

1. צרו דוח HTML של 10 התהליכים הכבדים (ConvertTo-Html) עם כותרת ו-CSS פשוט, שמרו לקובץ ופתחו בדפדפן (Invoke-Item).
2. ייצאו טבלה ל-HTML כ-HTML-Fragment -ושלבו שני Fragments (תהליכים + שירותים עצורים) בדוח אחד.

3. דחסו את כל הקבצים שיצרתם בתרגילים לקובץ lab.zip (Compress-Archive), ופרסו אותו לתיקייה חדשה (Expand-Archive).

4. בדקו את ה-Hash של קובץ ה-ZIP (Get-FileHash) לפני ואחרי העתקה.

4.7 מאתגר תרגיל מסכם – ממיר פורמטים

כתבו פונקציה Convert-DataFile עם הפרמטרים -Path -To -I (Html, Xml, Json, Csv : ValidateSet).

- הפונקציה מזהה את פורמט הקלט לפי הסיומת (Clixml של .xml / .json / .csv).
- ממירה לאובייקטים ושומרת בפורמט היעד באותה תיקייה עם אותו שם וסיומת חדשה.
- מחזירה את אובייקט הקובץ החדש (Get-Item).
- אם הקלט והפלט באותו פורמט – זורקת שגיאה ברורה.
- תומכת ב-WhatIf (SupportsShouldProcess).

בדקו: users.csv → json → xml → csv וודאו שהנתונים זהים בסוף.

כתבו סקריפט `Get-FolderReport.ps1` שמקבל נתיב תיקייה ומייצר **שלושה** קבצים:

1. `report.csv` – כל הקבצים: שם, סיומת, גודל ב-KB, תאריך שינוי אחרון.
 2. `summary.json` – לכל סיומת: כמות קבצים וגודל כולל, וגם 5 הקבצים הגדולים ביותר.
 3. `report.html` – דוח קריא שמשלב את שניהם.
- דרישות: פרמטרים עם ולידציה, `-Recurse` כ-Switch, `Write-Verbose` לכל שלב, טיפול בשגיאות גישה (-)
`Get-ChildItem C:\ -Directory | Get-Pipeline-Item` (עבודה נכונה מה-Pipeline), `(try/catch + ErrorAction`
`FolderReport`).